

# 1 Аннотация

Проблема пропущенных значений является одной из наиболее распространенных в анализе данных и машинном обучении, создавая значительные сложности на этапе предобработки. В данной работе представлено сравнительное исследование эффективности различных методов заполнения пропусков в табличных данных. Целью исследования была оценка качества восстановления данных с использованием как относительно простых подходов, таких как например заполнение средним значением, так и более сложных алгоритмов: **KNNImputer**, **MICE (Multiple Imputation by Chained Equations)** и **missForest**. На основе выбранных метрик качества производился анализ влияния каждого метода заполнения на итоговую структуру и репрезентативность данных. Результаты исследования позволяют дать рекомендации по выбору оптимальной стратегии заполнения пропущенных значений в зависимости от характера табличных данных и поставленных задач.

## 2 Введение

Наличие пропущенных значений в табличных данных — ситуация, когда некоторые ячейки данных не содержат информации, — представляет собой практически неизбежную проблему на этапе предобработки в задачах анализа данных и машинного обучения. Формально пропущенные данные определяются как отсутствие значения для переменной в наблюдении, которое может быть представлено специальными маркерами (NULL, NaN, "?"). Их появление обусловлено по нескольким причинам: отказы оборудования при сборе сенсорных данных, нежелание респондентов отвечать на конфиденциальные вопросы в социологических опросах, ошибки при ручном вводе информации или технические сбои при интеграции данных из разных источников. Независимо от причины, игнорирование проблемы пропусков или их некорректная обработка ведут к снижению статистической мощности анализа, смещению оценок и потере качества моделей. Есть несколько подходов, которые позволяют избежать этапа заполнения пропусков:

1. **Удаление строк, содержащие пропуски.** Несмотря на кажущуюся простоту, этот подход критикуется в литературе как крайне неэффективный: он приводит к значительной потере информации и, что важнее, вносит систематическое смещение, если пропуски не являются абсолютно случайными [1, 2]. Удаление допустимо лишь в редких случаях, соответствующих строгому условию MCAR (Missing Completely At Random), при минимальном проценте пропусков.

2. **Использование моделей, способных обучаться на данных с пропусками.** Некоторые алгоритмы машинного обучения обладают встроенными механизмами для работы с неполными данными. Например, алгоритмы построения деревьев решений (CART) могут использовать суррогатные разбиения (surrogate splits), подбирая коррелированный признак для замены основного условия при встрече с пропуском. Однако эти механизмы зачастую неэффективны. Они создают значительные вычислительные издержки, а их качество зависит от наличия стабильных и сильных корреляций между признаками, что на практике выполняется не всегда [3]. Современные градиентные бустинговые фреймворки (например, LightGBM) в своей документации прямо указывают, что явное предварительное заполнение пропусков зачастую даёт более предсказуемые и качественные результаты, чем на встроенной обработке.

Таким образом, ни удаление строк, ни использование встроенных механизмов алгоритмов не являются удовлетворительными решениями. Это обуславливает необходимость в применении специализированных, робастных методов заполнения пропусков, которые становятся важным этапом в обработке данных. Данная работа посвящена сравнительному анализу эффективности современных методов заполнения табличных пропусков, таких как KNNImputer, MICE и missForest, с целью выявления оптимальных стратегий для данных различной природы.

## 3 Методы оценки качества заполнения пропусков

Мы работали с большим набором различных заполненных датасетов, искусственно вносили пропуски тремя методами: **MAR**, **MNAR**, **MCAR** с 5, 15, 30, 50 процентами пропусков. На данный момент не существует общепринятого подхода для измерения качества заполнения пропусков. В этой статье мы рассматриваем 3 основных подхода измерения качества: (1) меры несоответствия между заполненными и истинными значениями отдельных образцов; (2) меры несоответствия в распределениях отдельных признаков для заполненных и истинных данных; (3) меры несоответствия для заполненных и истинных данных по всему распределению данных.

(1) В этой статье мы рассматриваем 3 метрики качества, которые явно сравнивают заполненные и истинные значения: среднеквадратическая ошибка (RMSE) [4] - [15], средняя абсолютная ошибка (MAE) [4], [8] - [10] и коэффициент детерминации ( $R^2$ ) [5, 6, 16]. Эти метрики оценивают точечную близость заполненных значений к исходным, но игнорируют восстановление исходного распределения данных. Как показано в исследовании [17], методы заполнения могут демонстрировать хорошие значения RMSE, MAE

или  $R^2$ , при этом существенно искажая многомерное распределение данных. В результате модели, обученные на таких данных, могут давать непредсказуемые результаты, некорректную интерпретацию важности признаков и сниженную обобщающую способность.

(2) Thurow и др. [13] рассматривают несколько мер распределения на основе признаков, включая дивергенцию Kullback–Leibler (KL), статистику Kolmogorov–Smirnov (KS) для двух выборок и расстояние 2-Wasserstein (2W). Эти метрики оценивают распределения только по каждому признаку отдельно (маргинальные распределения), но игнорируют совместные распределения между признаками — то есть многомерную структуру данных.

Все недостатки метрик (1), (2) исследованы в работе Tolou Shadbahr и др. [18], в этой же работе они вводят новую метрику, которая соответствует подходу (3), она называется sliced Wasserstein и она уже учитывает совместное распределение признаков.

## 4 Теоретические основы анализа неполных данных

### 4.1 Контекст и постановка проблемы

До формализации, предложенной Рубином [17], стандартные методы статистического вывода, включая оценку параметров и проверку гипотез, подразумевали наличие полных наборов данных. При наличии пропущенных значений практики либо исключали неполные наблюдения (что приводило к потере информации и потенциальному смещению оценок), либо применяли эвристические методы заполнения, такие как подстановка среднего или регрессионные оценки, без учёта неопределённости, порождаемой самим процессом импутации.

Целью работы Рубина было создание общей теории статистического вывода в условиях неполных данных, которая позволила бы получать корректные и несмещённые оценки параметров, а также чётко определять границы применимости различных методов.

Ключевым вкладом данной работы является формальное определение так называемых *механизмов пропусков* (missing data mechanisms). Эти механизмы описывают вероятностную природу возникновения пропусков, и от их типа напрямую зависит корректность и обоснованность любого последующего анализа.

### 4.2 Формальные определения типов пропусков (Механизмы отсутствия данных)

Рубин вводит формальное описание через *индикаторную матрицу пропусков*  $M$ . Для каждого элемента данных  $Y_{ij}$ , где  $i$  обозначает объект, а  $j$  — переменную, индикатор  $M_{ij} = 1$ , если значение пропущено, и  $M_{ij} = 0$  в противном случае.

Вся доступная для анализа информация представлена парой  $(Y_{\text{obs}}, M)$ , где  $Y_{\text{obs}}$  — множество наблюдаемых значений.

Механизм пропусков определяется условным распределением

$$P(M \mid Y, \phi),$$

где  $Y = (Y_{\text{obs}}, Y_{\text{mis}})$  — полный (но частично ненаблюдаемый) набор данных, а  $\phi$  — вектор параметров, управляющих процессом возникновения пропусков. Классификация Рубина основана на зависимости этого распределения от  $Y$ .

#### 4.2.1 Тип 1: Missing Completely At Random (MCAR)

*Формальное определение:*

$$P(M \mid Y, \phi) = P(M \mid \phi) \quad \text{для всех } Y.$$

Вероятность пропуска значения не зависит ни от наблюдаемой ( $Y_{\text{obs}}$ ), ни от пропущенной ( $Y_{\text{mis}}$ ) частей данных.

*Смысл:* Возникновение пропуска является случайным событием, не связанным с содержанием данных. Пропущенные значения формируют случайную подвыборку из генеральной совокупности.

*Примеры:*

- Случайное повреждение биологического образца при транспортировке в лабораторию.
- Технический сбой измерительного прибора, произошедший в случайный момент времени.
- Ошибка интервьюера, случайно пропустившего вопрос в анкете.

### 4.2.2 Тип 2: Missing At Random (MAR)

*Формальное определение:*

$$P(M | Y, \phi) = P(M | Y_{\text{obs}}, \phi) \quad \text{для всех } Y_{\text{mis}}.$$

Вероятность пропуска может зависеть от наблюдаемых данных ( $Y_{\text{obs}}$ ), но не зависит от самих пропущенных значений ( $Y_{\text{mis}}$ ) при условии учёта наблюдаемых переменных.

*Смысл:* Причина пропуска носит систематический характер, который может быть объяснён через другие наблюдаемые переменные в наборе данных.

*Примеры:*

- В медицинском исследовании пожилые пациенты (наблюдаемый возраст) чаще отказываются от сложного теста. Однако среди пациентов одного возраста вероятность отказа от теста не связана с его реальным, но неизвестным результатом.
- В социальном опросе респонденты с низким заявленным доходом чаще уклоняются от ответа на вопрос о сбережениях. При этом среди респондентов с одинаковым доходом склонность к уклонению не зависит от фактической суммы сбережений.

### 4.2.3 Тип 3: Missing Not At Random (MNAR)

*Формальное определение:* Распределение  $P(M | Y, \phi)$  зависит от пропущенных значений  $Y_{\text{mis}}$ , даже после учёта всех наблюдаемых данных  $Y_{\text{obs}}$ .

*Смысл:* Вероятность того, что значение окажется пропущенным, напрямую зависит от величины самого этого (неизвестного) значения. Это наиболее сложный для анализа тип пропусков.

*Примеры:*

- В опросе о доходах лица с очень высокими доходами намеренно скрывают эту информацию, отказываясь отвечать.
- В клинических испытаниях пациенты, испытывающие серьёзные побочные эффекты от препарата, с большей вероятностью прекращают участие в исследовании.
- Прибор для измерения уровня токсинов перестаёт передавать данные, когда концентрация превышает критический порог.

## 5 Методы заполнения пропусков

### 5.1 KNNImputer

Алгоритм KNNImputer (k-nearest neighbors imputer) основан на принципе подобию объектов в метрическом пространстве. Для каждого объекта  $i$ , содержащего пропущенные значения, определяются  $k$  ближайших соседей среди объектов с полными или частично заполненными данными. Близость вычисляется только по тем признакам, которые наблюдаются у обоих объектов (например, с использованием евклидовой метрики). Пропущенное значение в объекте  $i$  оценивается как средневзвешенное соответствующих значений у выбранных соседей, где веса могут зависеть от расстояния (ближайшие соседи вносят больший вклад). Этот метод, впервые применённый для обработки данных ДНК-микрочипов [19], хорошо работает, когда данные имеют локальную структуру сходства, но требует тщательного выбора параметра  $k$  и может быть вычислительно затратным для больших объёмов данных.

### 5.2 MICE (Multivariate Imputation by Chained Equations)

MICE — итеративный метод многократного заполнения, основанный на предположении, что условное распределение каждого признака с пропусками может быть смоделировано с помощью регрессии на остальные признаки. Процесс состоит из следующих шагов:

1. Исходные пропуски заполняются простыми методами (например, средним или случайным значением).
2. Для каждого признака с пропусками строится регрессионная модель (линейная, логистическая и др.) с использованием текущих заполненных значений остальных признаков.
3. Пропущенные значения заменяются на предсказания модели с добавлением случайного остатка для учёта неопределённости.
4. Шаги 2–3 повторяются заданное число итераций до стабилизации результатов.

Как отмечается в работе [20], MICE позволяет получать несмещённые оценки при условии правильной спецификации моделей и хорошо подходит для данных со сложными взаимосвязями между переменными. Метод также даёт возможность оценивать вариативность заполнения путём создания нескольких заполненных наборов данных.

### 5.3 MissForest

MissForest — непараметрический метод заполнения, основанный на использовании случайных лесов (Random Forest). В отличие от MICE, где для каждого признака строится отдельная регрессионная модель, MissForest применяет ансамбли решающих деревьев, что позволяет учитывать нелинейные зависимости и взаимодействия между признаками без предположений о форме распределения данных. Алгоритм работает итеративно:

1. Пропуски инициализируются средними/модами.
2. Признаки упорядочиваются по возрастанию доли пропущенных значений.
3. Для каждого признака строится модель случайного леса, предсказывающая его значения по остальным признакам.
4. Пропуски заполняются предсказанными значениями.
5. Итерации продолжаются до достижения критерия остановки (например, малое изменение в заполненных значениях между итерациями).

Согласно исследованию [21], MissForest демонстрирует высокую точность восстановления как для непрерывных, так и для категориальных переменных, устойчив к выбросам и сохраняет сложные структуры данных, однако требует значительных вычислительных ресурсов при большом количестве признаков.

### 5.4 Константные методы.

В исследовании также рассматриваются ещё 3 метода заполнения пропусков. Для категориальных признаков использовался следующий метод: 1) MODE, каждый пропуск в конкретном столбце для категориального признака заполнялся модой по этому столбцу. Для непрерывных признаков: 1) MEAN, каждый пропуск в конкретном столбце для непрерывного признака заполнялся средним значением по этому столбцу. 2) MEDIAN, каждый пропуск в конкретном столбце для непрерывного признака заполнялся медианой по этому столбцу. 3) NUL, каждый пропуск в конкретном столбце для непрерывного признака заполнялся нулем.

Программу, в которой реализованы методы заполнения пропусков можно посмотреть здесь: [Репозиторий на GitHub](#)

## 6 Турнир методов заполнения пропусков

### 6.1 Методология ранжирования методов заполнения

Для сравнения методов заполнения пропусков мы использовали подход, предложенный в рамках ежегодного соревнования WMT22 Metrics Shared Task [22]. В основе этого подхода лежит идея **турнирного ранжирования** с учётом статистической неразличимости результатов.

#### 6.1.1 Турнирное ранжирование внутри задачи

Пусть для некоторого датасета  $d$  (с фиксированным типом пропусков) имеется множество методов заполнения  $\mathcal{M} = \{m_1, \dots, m_k\}$ , каждый из которых получил числовую оценку качества  $s(m_i)$  (например,  $F1$ -меру). Чем выше значение  $s(m_i)$ , тем лучше работает метод.

Традиционный подход к ранжированию предполагает упорядочивание методов строго по убыванию  $s(m_i)$ . Однако в реальных задачах различия между методами могут быть статистически незначимыми. Для учёта этого факта мы используем **групповое ранжирование**:

1. Методы сортируются по убыванию  $s(m_i)$ .
2. Задаётся порог  $\varepsilon > 0$  (в наших экспериментах  $\varepsilon = 0.01$ , что соответствует 1% разницы в качестве). Два метода считаются статистически неразличимыми, если

$$s(m_i) - s(m_j) < \varepsilon.$$

3. Все методы, попавшие в одну группу неразличимости, получают **одинаковый ранг**. Если метод  $m_i$  значимо лучше  $m_j$  (разница  $\geq \varepsilon$ ), то он занимает более высокое место.

Таким образом, для каждой задачи (строки в таблице результатов) формируется вектор рангов  $\mathbf{r}_d = (r_d(m_1), \dots, r_d(m_k))$ , где меньшее значение ранга соответствует лучшему методу. Методы, показавшие близкие результаты, получают равные ранги, что позволяет избежать ложного впечатления о превосходстве при незначительных различиях.

### 6.1.2 Агрегация рангов по всем задачам

После того как для каждой из  $N$  задач получены ранги методов, необходимо вычислить итоговую оценку каждого метода по всем задачам. Простое усреднение рангов

$$\bar{r}(m_i) = \frac{1}{N} \sum_{d=1}^N r_d(m_i)$$

подразумевает, что все задачи вносят равный вклад в финальный рейтинг. Однако в реальных приложениях разные типы задач могут иметь различную важность.

В работе [22] используется **иерархическое взвешивание**, при котором общий вес распределяется между уровнями параметров (языковая пара, домен, уровень оценки и т.д.) равномерно на каждом уровне иерархии. Адаптируя этот принцип к нашей задаче, мы ввели веса для двух уровней:

- **Уровень 1: тип пропусков** (MCAR, MAR, MNAR) и процент пропусков (5%, 15%, 30%, 50%). Более сложные сценарии (например, MNAR\_50) получают больший вес.
- **Уровень 2: тип датасета** (бинарная классификация, мультикласс, регрессия). Задачи регрессии считаются более сложными для заполнения пропусков и поэтому имеют повышенный вес.

Итоговый вес  $w_d$  для задачи  $d$  вычисляется как произведение нормированных весов на каждом уровне:

$$w_d = \tilde{w}_{\text{type}}(d) \cdot \tilde{w}_{\text{dataset}}(d),$$

где нормировка выполнена так, чтобы сумма весов на каждом уровне равнялась заданной доле (в нашем случае 0.5 на уровень, то есть 50% важности отводится типу пропусков и 50% — типу датасета). Финальная нормализация обеспечивает  $\sum_{d=1}^N w_d = 1$ .

После этого вычисляется **средневзвешенный ранг** для каждого метода:

$$\bar{r}_w(m_i) = \sum_{d=1}^N w_d \cdot r_d(m_i).$$

### 6.1.3 Обоснование корректности взвешенного ранжирования

Корректность использования средневзвешенного ранга как агрегированной меры качества обоснована в работе [23]. В ней показано, что агрегация рангов с весами эквивалентна поиску **Кемени-оптимального консенсуса** (Kemeny consensus) — ранжирования, которое минимизирует суммарное расстояние Кендалла до ранжирований отдельных задач. Такой подход обладает рядом преимуществ:

- Он устойчив к различиям в шкалах оценок между разными задачами.
- Позволяет учесть разную важность задач через веса.
- Удовлетворяет аксиомам теории общественного выбора (отсутствие диктатора, монотонность и др.).

Таким образом, используемая нами процедура — турнирное ранжирование внутри задачи с последующим иерархически взвешенным усреднением — полностью соответствует современной методологии оценки систем машинного обучения, предложенной в ведущих работах 2022 года.

### 6.1.4 Итоговый рейтинг

Финальный рейтинг методов получается сортировкой по возрастанию  $\bar{r}_w(m_i)$ .

## 7 Датасеты

В эксперименте использовался набор из 96 датасетов, которые есть в открытом доступе, из ни 44 - для задачи регрессии, 52 - классификации. Мы искусственно внесли в эти датасеты пропуски трех типов: MAR, MCAR, MNAR. В каждом из случаев рассмотрели разное количество процентов пропусков  $p = 5, 15, 30, 50$ . Таким образом, для каждого исходного датасета мы получили 12 датасетов с пропусками.

На рисунках (1) – (2) представлены некоторые свойства этих датасетов.

| Data set                                        | Instances | Attributes | Classes |
|-------------------------------------------------|-----------|------------|---------|
| 2dplanes_binclass_binaryClass.csv               | 40768     | 10         | 2       |
| abalone_binclass_binaryClass.csv                | 4177      | 9          | 2       |
| aileron_binclass_binaryClass.csv                | 13750     | 40         | 2       |
| analcata_data_supreme_binclass_binaryClass.csv  | 4052      | 7          | 2       |
| bank8FM_binclass_binaryClass.csv                | 8192      | 8          | 2       |
| BNG(breast-w)_binclass_Class.csv                | 39366     | 9          | 2       |
| coil2000_regression_CARAVAN.csv                 | 9822      | 85         | 2       |
| cpu_act_binclass_binaryClass.csv                | 8192      | 21         | 2       |
| cpu_small_binclass_binaryClass.csv              | 8192      | 12         | 2       |
| delta_aileron_binclass_binaryClass.csv          | 7129      | 5          | 2       |
| delta_elevators_binclass_binaryClass.csv        | 9517      | 6          | 2       |
| electricity_binclass_class.csv                  | 45312     | 13         | 2       |
| fri_c0_1000_5_binclass_binaryClass.csv          | 1000      | 5          | 2       |
| fri_c1_1000_5_binclass_binaryClass.csv          | 1000      | 5          | 2       |
| fri_c3_1000_10_binclass_binaryClass.csv         | 1000      | 10         | 2       |
| fri_c3_1000_25_binclass_binaryClass.csv         | 1000      | 25         | 2       |
| fri_c3_1000_50_binclass_binaryClass.csv         | 1000      | 50         | 2       |
| fri_c3_1000_5_binclass_binaryClass.csv          | 1000      | 5          | 2       |
| fri_c4_1000_100_binclass_binaryClass.csv        | 1000      | 100        | 2       |
| fri_c4_1000_10_binclass_binaryClass.csv         | 1000      | 10         | 2       |
| fri_c4_1000_25_binclass_binaryClass.csv         | 1000      | 25         | 2       |
| fri_c4_1000_50_binclass_binaryClass.csv         | 1000      | 50         | 2       |
| isolet_multiclass_class.csv                     | 7797      | 617        | 26      |
| JapaneseVowels_multiclass_speaker.csv           | 9961      | 14         | 9       |
| kin8nm_binclass_binaryClass.csv                 | 8192      | 8          | 2       |
| letter_multiclass_class.csv                     | 20000     | 16         | 26      |
| mammography_binclass_class.csv                  | 11183     | 6          | 2       |
| meta_stream_intervals.arff_multiclass_class.csv | 45164     | 74         | 11      |
| mfeat-factors_multiclass_class.csv              | 2000      | 216        | 10      |
| mfeat-fourier_multiclass_class.csv              | 2000      | 76         | 10      |
| mfeat-karhunen_multiclass_class.csv             | 2000      | 64         | 10      |
| mfeat-morphological_multiclass_class.csv        | 2000      | 6          | 10      |
| mfeat-zernike_multiclass_class.csv              | 2000      | 47         | 10      |
| mnist_784_multiclass_class.csv                  | 70000     | 784        | 10      |
| optdigits_multiclass_class.csv                  | 5620      | 64         | 10      |
| page-blocks_multiclass_class.csv                | 5473      | 10         | 5       |
| pendigits_multiclass_class.csv                  | 10992     | 16         | 10      |
| pol_binclass_binaryClass.csv                    | 15000     | 48         | 2       |
| pol_regression_foo.csv                          | 15000     | 48         | 11      |
| puma32H_binclass_binaryClass.csv                | 8192      | 32         | 2       |
| puma8NH_binclass_binaryClass.csv                | 8192      | 8          | 2       |
| quake_binclass_binaryClass.csv                  | 2178      | 3          | 2       |
| satellite_image_regression_class.csv            | 6435      | 36         | 6       |
| satimage_multiclass_class.csv                   | 6430      | 36         | 6       |
| scene_binclass_Urban.csv                        | 2407      | 299        | 2       |
| segment_multiclass_class.csv                    | 2310      | 19         | 7       |
| space_ga_binclass_binaryClass.csv               | 3107      | 6          | 2       |
| spambase_binclass_class.csv                     | 4601      | 57         | 2       |
| waveform-5000_multiclass_class.csv              | 5000      | 40         | 3       |
| wine_quality_regression_quality.csv             | 6497      | 11         | 7       |
| yeast_ml8_binclass_class14.csv                  | 2417      | 116        | 2       |
| yeast_multiclass_class_protein_localization.csv | 1484      | 8          | 10      |

(1) задача классификации

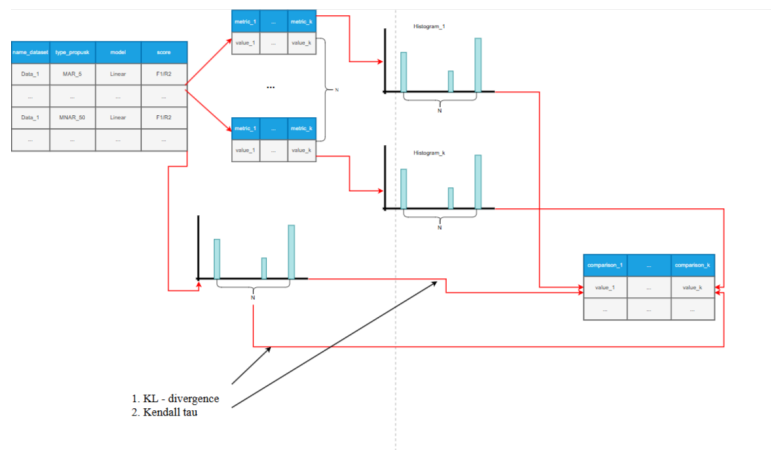
| Data set                                        | Instances | Attributes |
|-------------------------------------------------|-----------|------------|
| 2dplanes_regression_y.csv                       | 40768     | 10         |
| Ailerons_regression_goal.csv                    | 13750     | 40         |
| analcatdata_supreme_regression_Log_exposure.csv | 4052      | 7          |
| bank32nh_regression_rej.csv                     | 8192      | 32         |
| bank8FM_regression_rej.csv                      | 8192      | 8          |
| cpu_act_regression_usr.csv                      | 8192      | 21         |
| cpu_small_regression_usr.csv                    | 8192      | 12         |
| delta_elevators_regression_Se.csv               | 9517      | 6          |
| elevators_regression_Goal.csv                   | 16599     | 18         |
| fried_regression_Y.csv                          | 40768     | 10         |
| fri_c0_1000_10_regression_oz11.csv              | 1000      | 10         |
| fri_c0_1000_25_regression_oz26.csv              | 1000      | 25         |
| fri_c0_1000_50_regression_oz51.csv              | 1000      | 50         |
| fri_c0_1000_5_regression_oz6.csv                | 1000      | 5          |
| fri_c1_1000_10_regression_oz11.csv              | 1000      | 10         |
| fri_c1_1000_25_regression_oz26.csv              | 1000      | 25         |
| fri_c1_1000_50_regression_oz51.csv              | 1000      | 50         |
| fri_c1_1000_5_regression_oz6.csv                | 1000      | 5          |
| fri_c2_1000_10_regression_oz11.csv              | 1000      | 10         |
| fri_c2_1000_25_regression_oz26.csv              | 1000      | 25         |
| fri_c2_1000_50_regression_oz51.csv              | 1000      | 50         |
| fri_c2_1000_5_regression_oz6.csv                | 1000      | 5          |
| fri_c3_1000_10_regression_oz11.csv              | 1000      | 10         |
| fri_c3_1000_25_regression_oz26.csv              | 1000      | 25         |
| fri_c3_1000_50_regression_oz51.csv              | 1000      | 50         |
| fri_c3_1000_5_regression_oz6.csv                | 1000      | 5          |
| fri_c4_1000_100_regression_oz101.csv            | 1000      | 100        |
| fri_c4_1000_10_regression_oz11.csv              | 1000      | 10         |
| fri_c4_1000_25_regression_oz26.csv              | 1000      | 25         |
| fri_c4_1000_50_regression_oz51.csv              | 1000      | 50         |
| houses_regression_median_house_value.csv        | 20640     | 8          |
| house_16H_regression_price.csv                  | 22784     | 16         |
| house_8L_regression_price.csv                   | 22784     | 8          |
| kin8nm_regression_y.csv                         | 8192      | 8          |
| mtp_regression_oz203.csv                        | 4450      | 202        |
| pollen_regression_DENSITY.csv                   | 3848      | 4          |
| puma32H_regression_thetadd6.csv                 | 8192      | 32         |
| puma8NH_regression_thetadd3.csv                 | 8192      | 8          |
| quake_regression_col_4.csv                      | 2178      | 3          |
| quake_regression_richter.csv                    | 2178      | 3          |
| topo_2_1_regression_oz267.csv                   | 8885      | 266        |
| visualizing_soil_regression_track.csv           | 8641      | 4          |
| wind_regression_MAL.csv                         | 6574      | 14         |
| yprop_4_1_regression_oz252.csv                  | 8885      | 251        |

## (2) задача регрессии

Более детально ознакомиться с датасетами можно по ссылке: Репозиторий на GitHub

## 8 Турнир методов качества заполнения пропусков

Для анализа методов качества заполнения использовался турнир, описанный в статье Freitag et al. (2022)[22]. Ниже приведена схема подготовки к турниру:



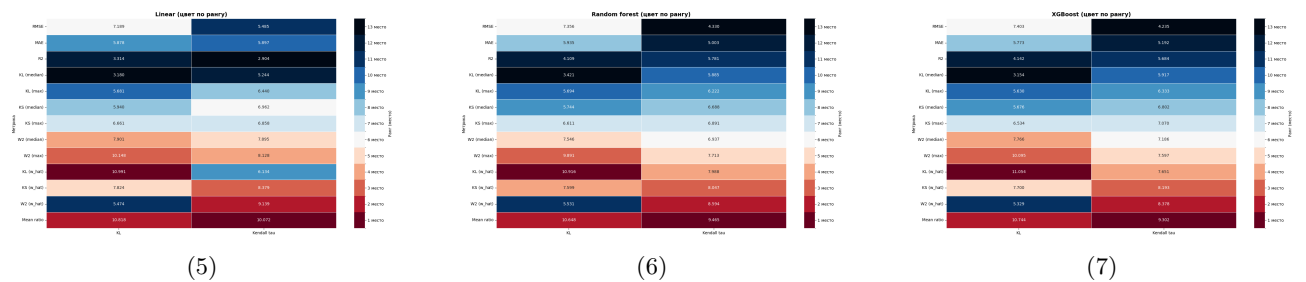
(4)

Проводилось три турнира для каждой из следующих моделей: 1) LinearRegression для задач регрессии, LogisticRegression для задач классификации. 2) RandomForestRegressor для задач регрессии, RandomForestClassifier для задач классификации. 3) XGBRegressor для задач регрессии, XGBClassifier для задач классификации.

Для каждой из приведенных выше моделей мы делали следующее:

- 1) Рассматривали каждый датасет из нашего общего набора;
- 2) Для него есть 12 типов пропусков, описанных выше ( $MAR\_5, \dots, MNAR\_50$ );
- 3) К имеющейся таблице приписывали столбец с метриками качества моделей в зависимости от задачи (регрессия -  $R2$ , классификация -  $F1$ );
- 4) Использовали  $N$  копий этой таблицы, где  $N$  - количество методов заполнения пропусков, и приписывали столбцы с метриками качества заполнения пропусков для каждого из методов заполнения. Количество этих метрик обозначили за  $k$ . Метрики качества заполнения были взяты из [18] и были описаны выше.
- 5) Для каждой из  $k$  метрик получили свое распределение, состоящее из  $N$  компонент. Посчитали  $KL$  — и  $Kendalltau$  для каждой из распределений с распределением метрик нашей модели (столбец с  $R2$ ,  $F1$ ). Получили  $k$  значений. Для них проводится турнир.

Ниже приведены результаты турнира для каждой из модели:



## 9 Метаобучение

Метаобучение (*meta-learning*) — это раздел машинного обучения, который позволяет моделям ИИ самостоятельно осваивать новые задачи. Главная цель метаобучения — научить машины тому, как учиться. Система накапливает опыт решения разных задач и использует его, чтобы быстрее и эффективнее осваивать новые.

Для сбора мета-данных использовался пакет *pymfe*. Более подробно можно прочесть здесь Репозиторий на GitHub.

Мы использовали два типа мета-признаков:

- **General:** Общая информация, относящаяся к набору данных, также известная как простые показатели, например количество экземпляров, атрибутов и классов;
- **Statistical:** Стандартные статистические показатели для описания числовых характеристик распределения данных.

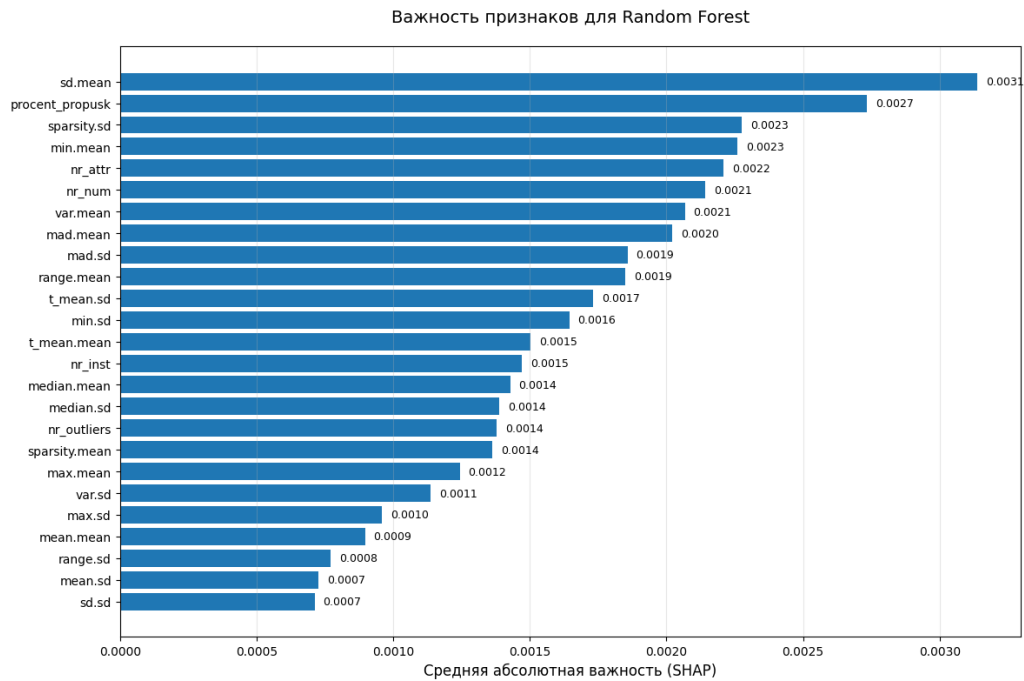
К этим признакам мы добавили еще признаки:

- **procent\_propusk** - возможные значения  $p = 5, 15, 30, 50$ ;
- **type\_propusk** - возможные значения : 1 – 12 ( $MAR\_5, \dots, MNAR\_5, \dots, MCAR\_50$ ).

Целевая переменная **method\_zapolnenia** принимает значения: 0–6 (соответственно MICE, MEAN\_MODE, missForest, MED\_MODE, NUL\_MODE, KNN15).

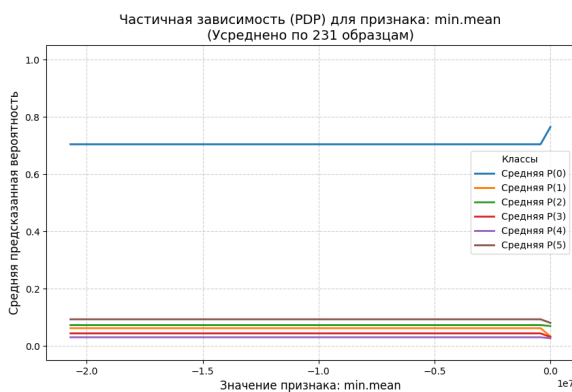
Используя библиотеку *shap*, определяем самые важные признаки, чтобы потом мы могли бы интерпретировать наши результаты. Ниже показаны важности признаков по убыванию:



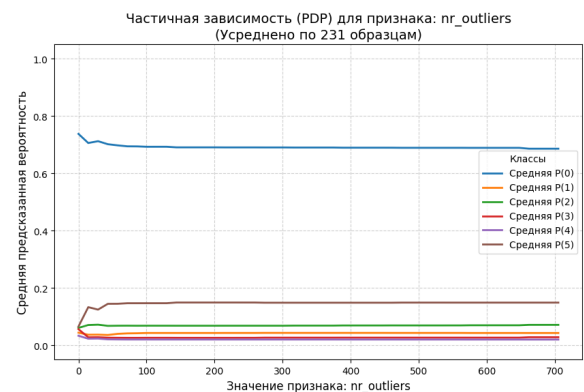


(8)

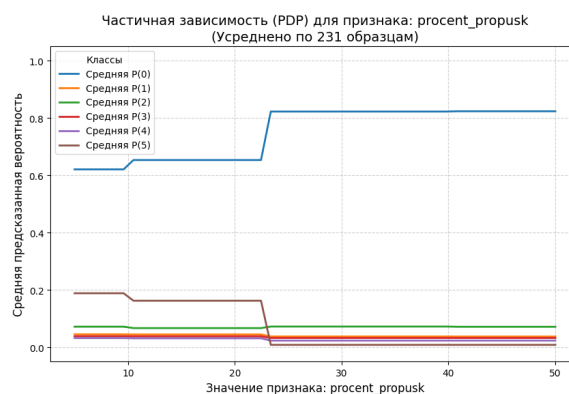
Затем мы варьируем 5 самых важных признаков и рисуем гистограммы по распределению вероятностей.



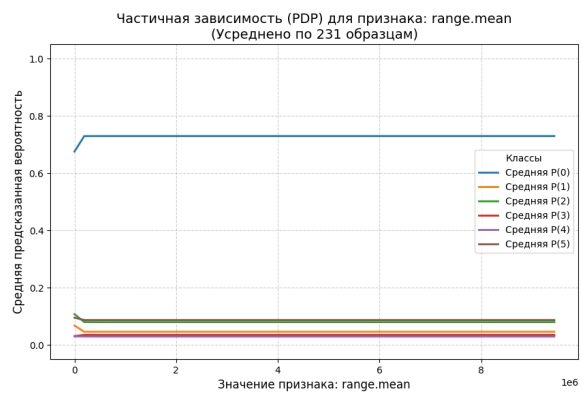
(9)



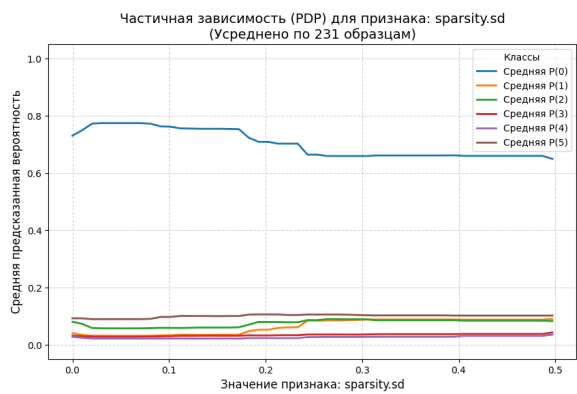
(10)



(11)



(12)



(13)

## Список литературы

- [1] Little, R. J., & Rubin, D. B. (2019). *Statistical analysis with missing data* (3rd ed.). John Wiley & Sons. (Теоретическое обоснование смещения и потери информации при удалении строк).
- [2] Schafer, J. L., & Graham, J. W. (2002). Missing data: our view of the state of the art. *Psychological methods*, 7(2), 147. (Эмпирическая демонстрация расточительности listwise deletion).
- [3] Twala, B. E. T. H., Jones, M. C., & Hand, D. J. (2008). Good methods for coping with missing data in decision trees. *Pattern Recognition Letters*, 29(7), 950–956. (Сравнительное исследование, показывающее неэффективность встроенных механизмов деревьев по сравнению с импутацией).
- [4] Luo, Y. Evaluating the state of the art in missing data imputation for clinical data. *Brief. Bioinforma.* **23**, bbab489 (2022).
- [5] Chang, C., Deng, Y., Jiang, X. & Long, Q. Multiple imputation for analysis of incomplete data in distributed health data networks. *Nat. Commun.* **11**, 5467 (2020).
- [6] Deng, Y., Chang, C., Ido, M. S. & Long, Q. Multiple imputation for general missing data patterns in the presence of high-dimensional data. *Sci. Rep.* **6**, 21689 (2016).
- [7] Schmitt, P., Mandel, J. & Guedj, M. A comparison of six methods for missing data imputation. *J. Biometrics Biostat.* **6**, <https://www.omicsonline.org/openaccess/a-comparison-of-six-methods-for-missing-data-imputation-2155-6180-1000224.php?aid=54590> (2015).
- [8] Muzellec, B., Josse, J., Boyer, C. & Cuturi, M. Missing data imputation using optimal transport. In *Proceedings of the 37th International Conference on Machine Learning*, 7130–7140 (PMLR, 2020).
- [9] Lin, W.-C. & Tsai, C.-F. Missing value imputation: a review and analysis of the literature (2006–2017). *Artif. Intell. Rev.* **53**, 1487–1509 (2020).
- [10] Platias, C. & Petasis, G. A comparison of machine learning methods for data imputation. In *11th Hellenic Conference on Artificial Intelligence, SETN 2020*, 150–159 (Association for Computing Machinery, Athens, Greece, 2020).
- [11] Armina, R., Zain, A. M., Ali, N. A. & Sallehuddin, R. A review on missing value estimation using imputation algorithm. *J. Phys. Conf. Ser.* **892**, 012004 (2017).
- [12] Thurow, M., Dumpert, F., Ramosaj, B. & Pauly, M. Imputing missings in official statistics for general tasks—our vote for distributional accuracy. *Stat. J. IAOS* **37**, 1379–1390 (2021).
- [13] Jäger, S., Allhorn, A. & Bießmann, F. A benchmark for data imputation methods. *Front. Big Data* **4**, <https://www.frontiersin.org/article/10.3389/fdata.2021.693674> (2021).
- [14] Zhou, Y.-H. & Saghapour, E. ImputeHR: a visualization tool of imputation for the prediction of biomedical data. *Front. Genet.* **12**, 691274 (2021).
- [15] Luo, Y. Evaluating the state of the art in missing data imputation for clinical data. *Brief. Bioinforma.* **23**, bbab489 (2022).
- [16] van Buuren, S. *Flexible Imputation of Missing Data*, 2nd edn. (CRC Press, 2018).
- [17] Donald B. Rubin. *Inference and Missing Data*. *Biometrika*, 63(3):581–592, 1976.
- [18] Shadbahr, T., Roberts, M., Stanczuk, J. et al. The impact of imputation quality on machine learning classifiers for datasets with missing values. *Commun Med* **3**, 139 (2023). <https://doi.org/10.1038/s43856-023-00356-z>
- [19] Olga Troyanskaya, Mike Cantor, Trevor Hastie, Rob Tibshirani, David Botstein, and Russ Altman. *Missing value estimation methods for dna microarrays*. *Bioinformatics*, 17:520–525, 07 2001.
- [20] Zhongheng Zhang. *Multiple imputation with multivariate imputation by chained equation (mice) package*. *Annals of Translational Medicine*, 4(2):30, 2016.
- [21] Daniel J. Stekhoven and Peter Bühlmann. *MissForest—non-parametric missing value imputation for mixed-type data*. *Bioinformatics*, 28(1):112–118, 2012.

- [22] Markus Freitag, Ricardo Rei, Nitika Mathur, et al. *Results of WMT22 Metrics Shared Task: Stop Using BLEU - Neural Metrics Are Better and More Robust*. In Proceedings of the Seventh Conference on Machine Translation (WMT), pages 46–68, 2022[cite: 730, 755].
- [23] Pierre Colombo, Nathan Noiry, Ekhine Irurozki, and Stéphan Cléménçon. *What are the best systems? New perspectives on NLP benchmarking*. arXiv preprint arXiv:2202.03799, 2022.